

Phase 03: Real Backend Data Systems

Task 08 — Required Interview Questions (Quick Reference Guide)

1. What is DbContext and what does it do in your project?

DbContext is the bridge between your C# application and the database. It manages database connections, tracks entity changes, executes queries, and saves changes to the database.

Example in Our Project:

In TrainingCenterDbContext, it connects our entities (Students, TrainingTracks, Enrollments, Payments) to SQL Server and saves changes using SaveChangesAsync().

2. What is DbSet and how does it map to database tables?

A DbSet<T> represents a specific table in the database that you can query and save data to using LINQ.

Example:

```
// DbSet<Student> maps to the [Students] table in SQL Server:
public DbSet<Student> Students => Set<Student>();

// Querying the table via DbSet:
var activeStudents = await _context.Students.Where(s => s.IsActive).ToListAsync();
```

3. What is a migration and why do we use it?

A migration is a code-first version control tool for your database schema. We use it to update and keep the database structure in sync with changes in our C# entity models over time without losing existing data.

Example:

```
dotnet ef migrations add AddStudentPhone
dotnet ef database update
```

4. What is the difference between entity and DTO?

- **Entity:** Internal model that maps directly to a database table and is tracked by EF Core.
- **DTO (Data Transfer Object):** Simple contract model used to receive request inputs or send response data over API endpoints without exposing database internals.

Example:

Student is the database entity with all columns and foreign keys, while *StudentResponse* is the DTO returning only formatted public fields to the client.

5. Why should APIs not return EF entities directly?

Returning entities directly causes:

1. **Security Risks:** Exposes internal columns (e.g., passwords, soft-delete flags).
 2. **Circular Reference Errors:** Causes JSON serializer loops on bi-directional relationships.
 3. **Tight Coupling:** Any database column change instantly breaks client applications.
-

6. What is a foreign key?

A foreign key is a column in a child table that points to the primary key of a parent table to enforce relational integrity and prevent orphan records.

Example:

TrainingTrack.PrimaryInstructorId is a foreign key referencing *Instructor.InstructorId*.

7. Explain the relationship between Student, TrainingTrack and Enrollment.

*It is a **Many-to-Many** relationship resolved through an explicit join entity (*Enrollment*):*

- One **Student** can have many **Enrollments**.
 - One **TrainingTrack** can have many **Enrollments**.
 - Each **Enrollment** links one student to one track and holds payment and progress records.
-

8. Why is Enrollment a join entity instead of a simple many-to-many?

Because an enrollment has its own data and lifecycle: `EnrollmentDate`, `Status` (Pending, Active, Completed, Cancelled), `ProgressPercentage`, and linked `Payments`. A simple many-to-many table can only hold the two IDs.

9. What is Include and when did you use it?

*`.Include()` is used for **Eager Loading** to fetch related entities in the same SQL query using a `JOIN`, avoiding lazy loading and `N+1` query issues.*

Example:

```
var enrollment = await _context.Enrollments
    .Include(e => e.Student)
    .Include(e => e.TrainingTrack)
    .Include(e => e.Payments)
    .FirstOrDefaultAsync(e => e.EnrollmentId == id);
```

10. What is Select projection and why is it useful?

`.Select()` projects only the required columns directly into a DTO. It is useful because it speeds up queries, reduces memory and network usage, and does not attach objects to the EF change tracker.

Example:

```
var students = await _context.Students
    .Select(s => new StudentResponse {
        StudentId = s.StudentId,
        FullName = s.FullName,
        Email = s.Email
    }).ToListAsync();
```

11. What is pagination and why does an API need it?

Pagination breaks large query results into smaller pages using `pageNumber` and `pageSize` via `Skip()` and `Take()`. An API needs it to protect server memory, prevent query timeouts, and keep response times fast.

Example:

```
var items = await query.Skip((pageNumber - 1) * pageSize).Take(pageSize).ToListAsync();
```

12. How did you prevent duplicate active enrollments?

In `EnrollmentService`, before creating an enrollment, we query the database to check if the student already has an `Active` or `Pending` enrollment in that track. If found, we throw a `BadRequestException` (HTTP 400).

Example:

```
var exists = await _context.Enrollments.AnyAsync(e =>
    e.StudentId == request.StudentId &&
    e.TrainingTrackId == request.TrainingTrackId &&
    (e.Status == EnrollmentStatus.Active || e.Status == EnrollmentStatus.Pending));

if (exists) throw new BadRequestException("Student is already enrolled in this track.");
```

13. How did you protect track capacity?

When enrolling a student, we count active and pending enrollments in the track (ignoring cancelled ones). If `activeCount >= track.Capacity`, the registration is rejected with HTTP 400.

Example:

```
var activeCount = track.Enrollments.Count(e => e.Status == EnrollmentStatus.Active || e.
    Status == EnrollmentStatus.Pending);
if (activeCount >= track.Capacity)
    throw new BadRequestException("Track has reached maximum capacity.");
```

14. How did you handle payment validation?

In `PaymentService`, we enforced three validations:

- 1. Payment amount must be positive (`Amount > 0`).*
 - 2. Reject payments on cancelled enrollments.*
 - 3. Reject overpayment: `Amount` cannot exceed $\text{Track.Price} - \sum \text{CompletedPayments}$.*
-

15. What is soft delete and why did you use it?

*Soft delete marks records with an `IsDeleted = true` flag instead of executing a SQL `DELETE`. We use it to preserve historical student and payment records while automatically hiding deleted records using EF Core **Global Query Filters**.*

Example:

```
// In StudentConfiguration.cs:
builder.HasQueryFilter(s => !s.IsDeleted);
```

16. What is the difference between local database and remote database?

- **Local Database:** *Runs locally on your machine (e.g., LocalDB) for development and offline testing.*
 - **Remote Database:** *Runs on a cloud server (e.g., Azure SQL) accessible over the network for multi-user production, with automated backups, high availability, and security.*
-

17. How did you configure the production connection string?

In production, connection strings are injected via secure Environment Variables or Azure Key Vault (never hardcoded), and configured in Program.cs with connection retry resilience (EnableRetryOnFailure).

18. Why should connection strings not be pushed to GitHub?

Because connection strings contain sensitive credentials (database host, username, and password). Exposing them on GitHub allows unauthorized attackers to steal, alter, or delete production data.

19. What is the hardest bug you faced in deployment?

Foreign key deletion restrictions and decimal rounding mismatches between the in-memory test database and real SQL Server. We fixed this by setting explicit Fluent API decimal precisions (`decimal(18,2)`) and `DeleteBehavior.Restrict`.

20. If you had one more week, how would you improve the system?

1. **Redis Caching:** *Cache heavy reports like Dashboard Summary for fast responses.*
2. **Background Jobs (Hangfire):** *Send automated email payment receipts asynchronously.*
3. **CQRS with MediatR:** *Separate read and write pipelines for cleaner architecture.*